Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Practica:

Sincronizacion

Profesora:

M.I. Elba Karen Saenz Garcia

Fecha de Entrega:

17 de Abril del 2026

Índice

1 Desarrollo: **1**
1.1 Algoritmo de Cristian: 1
1.1.1 Actividad 1 1
1.1.2 Actividad 2 3
1.2 Algoritmo de Berkeley: 9
1.2.1 Notas: 9
1.2.2 Actividad 3 10
1.2.3 Actividad 4 12
1.2.4 Actividad 5 14
1.3 Algoritmo de Lamport: 15
1.3.1 Actividad 6 15
1.3.2 Actividad 7 22

Referencias **28**

1. Desarrollo:

1.1. Algoritmo de Cristian:

1.1.1. Actividad 1

Antes de comenzar con la simulación de latencia, se modificaron ligeramente los códigos originales del cliente para mejorar la presentación de los resultados en la consola. Específicamente, se actualizó el bloque de impresiones (`print`) al final del script para tabular los datos, limitar la salida a 6 decimales (`:.6f`) y mostrar claramente cada una de las variables matemáticas que son parte del algoritmo (t_0, C_s, t_1, RTT y RTT/2).

El fragmento modificado para la salida en consola es el siguiente:

```python
# --- Salida en consola ---
print("\n" + "="*65)
print("   RESULTADOS DE SINCRONIZACION (CRISTIAN)")
print("="*65)
print(f"Tiempo de envio (t0):                    {t0:.6f}")
print(f"Tiempo del servidor (Cs):                {tiempoServi:.6f}")
print(f"Tiempo de recepcion (t1):                {t1:.6f}")
print("-" * 65)
print(f"RTT Tiempo de viaje total (t1 - t0):     {RTT:.6f} segundos")
print(f"Desfase calculado (RTT/2):               {RTT/2:.6f} segundos")
print("-" * 65)
print(f"Reloj actual del cliente:                {t1:.6f}")
print(f"NUEVO RELOJ AJUSTADO (C):                {C:.6f}")
print("="*65 + "\n")
```

Simular latencia en red agregando

```
delay = 2 # segundos
time.sleep(delay)
```

Este fragmento se implementó en el script del cliente en la siguiente ubicación exacta:

```python
# 2. Registrar el tiempo exacto antes de enviar la solicitud (t0)
t0 = time.time()
# Actividad 1: Simular latencia en red de 2 segundos
delay = 2 # segundos
time.sleep(delay)
# 3. Recibir el tiempo del servidor (Cs) y convertirlo a float
tiempoServi = float(client.recv(1024).decode())
```

La ubicación de este retardo (`time.sleep(2)`) se implementó en esa parte para que la simulación funcione correctamente. Al colocarlo justo después de haber guardado el tiempo de envío local (t_0) y antes de procesar la respuesta del servidor (C_s), se obliga al cliente a pausar su ejecución durante 2 segundos y con esto simulamos el comportamiento de una red con alta latencia donde el mensaje tarda más tiempo en viajar, lo que incrementará directamente el RTT (Round-Trip Time) calculado posteriormente.

¿Como cambia el ajuste del reloj?

Antes de responder a la pregunta debemos observar el comportamiento de la simulación mediante las salidas en consola, tanto en una ejecución normal (sin latencia) como en la simulación con latencia, como se muestra a continuación:

Salida en consola sin latencia:

```
================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN)
================================================================
Tiempo de envio (t0):                   1775026700.020524
Tiempo del servidor (Cs):               1775026700.020815
Tiempo de recepcion (t1):               1775026700.021159
----------------------------------------------------------------
RTT Tiempo de viaje total (t1 - t0):    0.000635 segundos
Desfase calculado (RTT/2):              0.000318 segundos
----------------------------------------------------------------
Reloj actual del cliente:               1775026700.021159
NUEVO RELOJ AJUSTADO (C):               1775026700.021132
================================================================
```

Salida en consola con latencia simulada (2 segundos):

```
================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN)
================================================================
Tiempo de envio (t0):                   1775026739.583974
Tiempo del servidor (Cs):               1775026739.584519
Tiempo de recepcion (t1):               1775026741.584483
----------------------------------------------------------------
RTT Tiempo de viaje total (t1 - t0):    2.000508 segundos
Desfase calculado (RTT/2):              1.000254 segundos
----------------------------------------------------------------
Reloj actual del cliente:               1775026741.584483
NUEVO RELOJ AJUSTADO (C):               1775026740.584774
================================================================
```

Al analizar ambas salidas, el cambio en el ajuste del reloj es evidente. En la ejecución sin latencia, el tiempo de viaje total (RTT) es de apenas unos microsegundos (0.000635 s), lo que resulta en un desfase prácticamente nulo. Por lo tanto, el nuevo reloj ajustado (C) es casi idéntico al reloj local actual del cliente.

Por el contrario, al introducir el retardo de 2 segundos, el RTT se dispara a ≈ 2.0005 segundos, y el algoritmo de Cristian asume un desfase simétrico para el viaje de regreso de ≈ 1.0002 segundos. Dado que el servidor guardó su tiempo (C_s) de forma inmediata tras recibir la petición, pero el cliente tardó 2 segundos en procesarla, el cálculo final del nuevo reloj (C) resulta ser un segundo menor que el tiempo local actual del cliente ($C < C_c$).

Específicamente, el tiempo calculado es `1775026740.58`, mientras que el reloj local ya avanzó hasta `1775026741.58`. Bajo estas condiciones, las reglas de sincronización nos dicen que el reloj se debe detener o se ralentizar gradualmente por la diferencia calculada ($C_c - C$ unidades) hasta lograr la coincidencia.

1.1.2. Actividad 2

Modificar la implementación anterior para que se pueda trabajar con varios clientes al mismo tiempo (máquinas diferentes, pueden ser físicas o virtuales y multithread) y se vea una diferencia de tiempo.

Para lograr que el servidor atienda a múltiples clientes de manera simultánea, se modificó el script original incorporando la librería `threading`. En lugar de procesar cada conexión de manera secuencial y bloqueante, se definió una función `manejar_cliente` y se instanció un nuevo hilo por cada conexión entrante en el ciclo principal.

Fragmento modificado en el Servidor (Multithread):

```python
import socket
import time
import threading # Libreria agregada para soporte multihilo

# Funcion que maneja la peticion de cada cliente de forma independiente
def manejar_cliente(conn, addr):
    print(f"[+] Hilo iniciado para conexion desde: {addr}")
    Cs = time.time()
    conn.send(str(Cs).encode())
    conn.close()
    print(f"[-] Tiempo enviado ({Cs}) a {addr}. Hilo terminado.\n")

# ... (Configuracion del socket) ...
server.listen(10) # Se permite una cola de hasta 10 conexiones simultaneas

while True:
```

```
    conn, addr = server.accept()
    # Se crea y arranca un nuevo hilo por cada cliente conectado
    hilo = threading.Thread(target=manejar_cliente, args=(conn, addr))
    hilo.start()
```

Por el lado del cliente, para generar una diferencia de tiempo visible entre cada ejecución, se sustituyó el retardo fijo de 2 segundos por un retardo dinámico utilizando la librería `random`.

Fragmento modificado en el Cliente (Latencia aleatoria):

```
import random

# Simulamos una latencia aleatoria para cada cliente (entre 0.1 y 2.5 segundos
    )
delay = random.uniform(0.1, 2.5)
time.sleep(delay)
```

¿Todos obtienen el mismo ajuste? ¿Por qué?

Para responder a esta pregunta, se ejecutó el servidor multihilo junto con tres instancias del cliente de forma casi simultánea.

Salidas del Servidor y 3 Clientes (Caso sin atraso local):

Salida del servidor:

```
================================================================
Servidor MULTIHILO de Cristian escuchando en puerto 5000...
================================================================
[+] Hilo iniciado para conexion desde: ('127.0.0.1', 48216)
[-] Tiempo enviado (1775028039.400636) a ('127.0.0.1', 48216). Hilo terminado.

[+] Hilo iniciado para conexion desde: ('127.0.0.1', 48230)
[-] Tiempo enviado (1775028040.8103802) a ('127.0.0.1', 48230). Hilo terminado.

[+] Hilo iniciado para conexion desde: ('127.0.0.1', 48234)
[-] Tiempo enviado (1775028042.0961967) a ('127.0.0.1', 48234). Hilo terminado.
```

Salida del cliente 1:

```
================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN V2) - CLIENTE 1
================================================================
Tiempo local actual (Cc/t1): 1775028040.311921
```

```
RTT (Tiempo de viaje total): 0.911910 segundos
Desfase calculado (RTT/2):   0.455955 segundos
Tiempo calculado (C):          1775028039.856591
----------------------------------------------------------------
[ALERTA] C < Cc (El tiempo calculado es MENOR al local)
        -> ACCION: El reloj local NO se retrasa bruscamente.
        -> Se debe RALENTIZAR el reloj por 0.455330 unidades.
```

Salida del cliente 2:

```
================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN V2) - CLIENTE 2
================================================================
Tiempo local actual (Cc/t1): 1775028043.593453
RTT (Tiempo de viaje total): 2.783776 segundos
Desfase calculado (RTT/2):   1.391888 segundos
Tiempo calculado (C):          1775028042.202268
----------------------------------------------------------------
[ALERTA] C < Cc (El tiempo calculado es MENOR al local)
        -> ACCION: El reloj local NO se retrasa bruscamente.
        -> Se debe RALENTIZAR el reloj por 1.391185 unidades.
```

Salida del cliente 3:

```
    ================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN V2) - CLIENTE 3
================================================================
Tiempo local actual (Cc/t1): 1775028044.711709
RTT (Tiempo de viaje total): 2.616132 segundos
Desfase calculado (RTT/2):   1.308066 segundos
Tiempo calculado (C):          1775028043.404263
----------------------------------------------------------------
[ALERTA] C < Cc (El tiempo calculado es MENOR al local)
        -> ACCION: El reloj local NO se retrasa bruscamente.
        -> Se debe RALENTIZAR el reloj por 1.307447 unidades.
```

Al analizar las salidas, es evidente que **no, ninguno de los clientes obtuvo el mismo ajuste**.

Esto sucede porque el algoritmo de Cristian depende en su totalidad del RTT (tiempo de viaje de los mensajes). Al haber introducido una latencia aleatoria para simular diferentes condiciones de red en cada cliente, el RTT varió (0.91s, 2.78s y 2.61s respectivamente). Por lo tanto, el desfase calculado ($RTT/2$) fue distinto para cada nodo, obligando a que cada cliente se ralentice por una cantidad de unidades diferente e independiente de los demás.

Agregar al código lo necesario para considerar cuando C > Cc o C < Cc.

Para cumplir con este punto, se agregó un bloque lógico `if / elif` al final del script del cliente. Este bloque evalúa la relación entre el nuevo tiempo calculado (C) y el tiempo local actual del cliente (C_c), calculando las diferencias exactas y determina la acción correcta según las reglas de sincronización.

Fragmento modificado en el Cliente (Lógica de evaluación):

```
# --- LOGICA DE AJUSTE ---
if C > Cc:
    diferencia = C - Cc
    print("[ATENCION] C > Cc (El tiempo calculado es MAYOR al local)")
    print("         -> ACCION: Se ADELANTA el reloj local a la nueva hora.")
    print(f"        -> El salto hacia adelante es de {diferencia:.6f} segundos.")
elif C < Cc:
    diferencia = Cc - C
    print("[ATENCION] C < Cc (El tiempo calculado es MENOR al local)")
    print(f"         -> ACCION: El reloj local se debe ralentizar gradualmente.")
    print(f"         -> Se debe RALENTIZAR el reloj por {diferencia:.6f} unidades.")
else:
    print("[LISTO] C == Cc (El reloj ya esta sincronizado).")
```

Como se vio en la prueba anterior, si los relojes locales ya están en hora con el servidor pero hay latencia de red, siempre se da el caso de $C < C_c$. Para demostrar que el código también considera y evalúa correctamente el caso opuesto ($C > C_c$), se implementó un mecanismo para simular que el cliente tiene su reloj interno atrasado 10 segundos respecto a la realidad.

Para simular el caso normal sin retraso, se utilizaron las siguientes líneas:

```
t0 = time.time()
Cc = time.time() # Caso sin atraso
```

Para forzar el escenario $C > C_c$, se comentaron las líneas anteriores y se descomentaron las siguientes:

```
# Caso: Simulamos un reloj atrasado
atraso_falso = 10 # El cliente cree que son 10 segundos antes
t0 = time.time() - atraso_falso
Cc = time.time() - atraso_falso # Caso con atraso simulado
```

Con este atraso simulado, el bloque condicional detecta que el tiempo calculado es mayor al local, ejecutando la acción de “Adelantar el reloj". A continuación se muestran las salidas de este segundo caso:

Salidas del Servidor y 3 Clientes (Caso con reloj atrasado simulado):

Salida del servidor:

```
================================================================
Servidor MULTIHILO de Cristian escuchando en puerto 5000...
================================================================
[+] Hilo iniciado para conexion desde: ('127.0.0.1', 33688)
[-] Tiempo enviado (1775028149.109033) a ('127.0.0.1', 33688). Hilo terminado.

[+] Hilo iniciado para conexion desde: ('127.0.0.1', 33702)
[-] Tiempo enviado (1775028150.400608) a ('127.0.0.1', 33702). Hilo terminado.

[+] Hilo iniciado para conexion desde: ('127.0.0.1', 57254)
[-] Tiempo enviado (1775028151.4355612) a ('127.0.0.1', 57254). Hilo terminado.
```

Salida del cliente 1:

```
================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN V2) - CLIENTE 1
================================================================
Tiempo local actual (Cc/t1): 1775028140.150724
RTT (Tiempo de viaje total): 1.042983 segundos
Desfase calculado (RTT/2):   0.521491 segundos
Tiempo calculado (C):          1775028149.630525
----------------------------------------------------------------
[ALERTA] C > Cc (El tiempo calculado es MAYOR al local)
        -> ACCION: Se ADELANTA el reloj local a la nueva hora.
        -> El salto hacia adelante es de 9.479800 segundos.
```

Salida del cliente 2:

```
================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN V2) - CLIENTE 2
================================================================
Tiempo local actual (Cc/t1): 1775028141.977979
RTT (Tiempo de viaje total): 1.577898 segundos
Desfase calculado (RTT/2):   0.788949 segundos
Tiempo calculado (C):          1775028151.189557
----------------------------------------------------------------
[ALERTA] C > Cc (El tiempo calculado es MAYOR al local)
        -> ACCION: Se ADELANTA el reloj local a la nueva hora.
        -> El salto hacia adelante es de 9.211578 segundos.
```

Salida del cliente 3:

```
==================================================================
    RESULTADOS DE SINCRONIZACION (CRISTIAN V2) - CLIENTE 3
==================================================================
Tiempo local actual (Cc/t1): 1775028142.441949
RTT (Tiempo de viaje total): 1.006896 segundos
Desfase calculado (RTT/2):   0.503448 segundos
Tiempo calculado (C):          1775028151.939009
------------------------------------------------------------------
[ALERTA] C > Cc (El tiempo calculado es MAYOR al local)
        -> ACCION: Se ADELANTA el reloj local a la nueva hora.
        -> El salto hacia adelante es de 9.497060 segundos.
```

1.2. Algoritmo de Berkeley:

1.2.1. Notas:

- Cada cliente inicia con un reloj distinto
- El servidor calcula un promedio
- Todos convergen a un mismo tiempo
- Ejecutar el programa y responder:
    1. **¿Cuál era el tiempo inicial de cada cliente?**
    2. **¿Cuál fue el promedio?**
    3. **¿A qué tiempo convergen?**

Salida del programa:

```
Servidor de Berkeley escuchando en el puerto 60001...
Conexion establecida con: ('127.0.0.1', 47948)
Conexion establecida con: ('127.0.0.1', 47954)
Conexion establecida con: ('127.0.0.1', 47968)
Tiempo recibido del cliente: 1774828614.6528187
Tiempo recibido del cliente: 1774828611.847464
Tiempo recibido del cliente: 1774828616.3387685
Tiempo del servidor: 1774828613.939467
Tiempo promedio calculado por el servidor es: 1774828614.1946297
Ajuste del servidor: [-0.458, 2.347, -2.144]

Cliente 1 - Tiempo local antes del ajuste: 1774828611.847464
Cliente 1 - Ajuste recibido: 2.347165584564209
Cliente 1 - Nuevo tiempo despues del ajuste: 1774828614.1946297

Cliente 2 - Tiempo local antes del ajuste: 1774828614.6528187
Cliente 2 - Ajuste recibido: -0.4581890106201172
Cliente 2 - Nuevo tiempo despues del ajuste: 1774828614.1946297

Cliente 3 - Tiempo local antes del ajuste: 1774828616.3387685
Cliente 3 - Ajuste recibido: -2.144138813018799
Cliente 3 - Nuevo tiempo despues del ajuste: 1774828614.1946297
```

Respuestas:

1. **¿Cuál era el tiempo inicial de cada cliente?**

Los tiempos iniciales de cada cliente fueron:

- Cliente 1: 1774828611.847464 (2.35 segundos atrás del promedio)
- Cliente 2: 1774828614.652818 (0.46 segundos adelante del promedio)
- Cliente 3: 1774828616.338768 (2.14 segundos adelante del promedio)

Cada cliente simula una variación aleatoria de ±5 segundos respecto al tiempo real del sistema.

2. **¿Cuál fue el promedio?**

   El tiempo promedio calculado por el servidor fue **1774828614.1946297**. Este valor se obtiene promediando los tiempos de los 3 clientes más el tiempo del servidor (1774828613.939467), resultando en: $(611.84 + 614.65 + 616.34 + 613.94)/4 = 614.19$ (últimos dígitos significativos).

3. **¿A qué tiempo convergen?**

   Todos los clientes convergen al tiempo promedio **1774828614.1946297**. Como se puede observar en la salida, después de aplicar los ajustes correspondientes, el "Nuevo tiempo después del ajuste" es idéntico para los tres clientes, demostrando que el algoritmo de Berkeley logra sincronizar exitosamente todos los relojes al mismo valor.

1.2.2. Actividad 3

Para esta actividad se modificó el programa original para soportar un número variable de clientes. Los cambios principales fueron: (1) parametrizar la función `servidorBerkeley` y `ejecutaCliente` para recibir el número de clientes como argumento, (2) agregar la opción `SO_REUSEADDR` al socket para evitar errores de "address already in use" entre ejecuciones, (3) usar puertos diferentes para cada prueba (60001, 60002, 60003) evitando conflictos, y (4) crear una función `ejecutarPrueba` que encapsula la ejecución completa de servidor y clientes para cada configuración.

Modificar el programa para trabajar con:

- 5, 10 y 20 clientes
- Respondera las preguntas:
  1. **¿Cómo cambia el tiempo promedio?**
  2. **¿El sistema sigue siendo estable?**

Salida del programa con 10 clientes:

```
============================================================
```

```
                      SERVIDOR BERKELEY
============================================================
| Numero de clientes esperados: 10
| Puerto: 60002
============================================================

[SERVIDOR] Tiempos recibidos de clientes:
[SERVIDOR]   Cliente 01: 1774828674.637297
[SERVIDOR]   Cliente 02: 1774828676.781445
[SERVIDOR]   Cliente 03: 1774828670.725160
[SERVIDOR]   Cliente 04: 1774828674.653284
[SERVIDOR]   Cliente 05: 1774828678.582883
[SERVIDOR]   Cliente 06: 1774828675.318715
[SERVIDOR]   Cliente 07: 1774828670.358184
[SERVIDOR]   Cliente 08: 1774828678.589895
[SERVIDOR]   Cliente 09: 1774828678.777744
[SERVIDOR]   Cliente 10: 1774828680.287497
[SERVIDOR]   Servidor:   1774828675.524798

[SERVIDOR] Tiempo promedio: 1774828675.839719

[SERVIDOR] AJUSTES CALCULADOS
[SERVIDOR]   Cliente 01: +1.202421 segundos
[SERVIDOR]   Cliente 02: -0.941726 segundos
[SERVIDOR]   Cliente 03: +5.114559 segundos
[SERVIDOR]   Cliente 04: +1.186435 segundos
[SERVIDOR]   Cliente 05: -2.743164 segundos
[SERVIDOR]   Cliente 06: +0.521003 segundos
[SERVIDOR]   Cliente 07: +5.481535 segundos
[SERVIDOR]   Cliente 08: -2.750176 segundos
[SERVIDOR]   Cliente 09: -2.938025 segundos
[SERVIDOR]   Cliente 10: -4.447779 segundos
[SERVIDOR]   Servidor:   +0.314920 segundos

[SERVIDOR] SINCRONIZACION COMPLETADA
```

Respuestas:

1. **¿Cómo cambia el tiempo promedio?**

   El tiempo promedio se vuelve más representativo del estado global del sistema conforme aumenta el número de clientes. Con 3 clientes, el promedio se ve más influenciado por valores extremos (outliers), mientras que con 10 o 20 clientes, el efecto de un reloj muy adelantado o atrasado se diluye. En las pruebas realizadas, el promedio tiende a acercarse más al tiempo real del servidor cuando hay más

participantes, ya que las variaciones aleatorias (±5 segundos) tienden a cancelarse estadísticamente.

2. **¿El sistema sigue siendo estable?**

   Sí, el sistema permanece estable con 5, 10 y 20 clientes. El algoritmo de Berkeley es inherentemente escalable porque: (1) el cálculo del promedio es una operación O(n) simple, (2) cada cliente solo requiere una conexión TCP al servidor, y (3) el intercambio de mensajes es mínimo (solicitud de tiempo, envío de tiempo, recepción de ajuste). En todas las pruebas, todos los clientes convergieron exitosamente al tiempo promedio calculado. La única consideración es el tiempo de espera del servidor para recibir todos los tiempos, que aumenta proporcionalmente con el número de clientes.

1.2.3. Actividad 4

Para simular la latencia de red, se agregó un retraso aleatorio en la función `ClienteBerkeley` usando `time.sleep(random.uniform(0, 3))`. Este retraso se aplica en dos momentos: (1) antes de enviar el tiempo local al servidor (simulando latencia de envío), y (2) antes de recibir el ajuste del servidor (simulando latencia de recepción). De esta manera, cada cliente puede experimentar una latencia total de 0 a 6 segundos, representando condiciones variables de red.

- Agrega un retraso aleatorio en los clientes:
  - time.sleep(random.uniform(0, 3))
- Responder a las preguntas:
  1. **¿La latencia afecta el resultado final?**
  2. **¿Por qué el algoritmo de Berkeley es sensible o no a este factor?**

Salida del programa con latencia simulada:

```
============================================================
ACTIVIDAD 4 - SIMULACION DE LATENCIA DE RED
============================================================
[CLIENTE 01] Tiempo local: 1774828782.780560 (var: -0.93s)
[CLIENTE 01] Latencia simulada: 2.674 seg
[CLIENTE 01] Ajuste: -0.138924 -> Nuevo tiempo: 1774828782.641636

[CLIENTE 02] Tiempo local: 1774828779.546950 (var: -4.37s)
[CLIENTE 02] Latencia simulada: 1.777 seg
[CLIENTE 02] Ajuste: +3.094686 -> Nuevo tiempo: 1774828782.641636

[CLIENTE 03] Tiempo local: 1774828783.538568 (var: -0.57s)
[CLIENTE 03] Latencia simulada: 0.232 seg
[CLIENTE 03] Ajuste: -0.896932 -> Nuevo tiempo: 1774828782.641636
```

```
[CLIENTE 04] Tiempo local: 1774828779.330222 (var: -4.99s)
[CLIENTE 04] Latencia simulada: 2.648 seg
[CLIENTE 04] Ajuste: +3.311414 -> Nuevo tiempo: 1774828782.641636

[CLIENTE 05] Tiempo local: 1774828783.458510 (var: -1.06s)
[CLIENTE 05] Latencia simulada: 2.423 seg
[CLIENTE 05] Ajuste: -0.816874 -> Nuevo tiempo: 1774828782.641636

[SERVIDOR] Tiempo promedio: 1774828782.641636
[SERVIDOR] SINCRONIZACION COMPLETADA
============================================================
```

Respuestas:

1. **¿La latencia afecta el resultado final?**

   No, la latencia **no afecta el resultado final** de la sincronización en términos de convergencia. Como se puede observar en la salida, a pesar de que cada cliente experimentó diferentes latencias (desde 0.232 hasta 2.674 segundos), todos convergieron al mismo tiempo promedio (1774828782.641636). Esto se debe a que el algoritmo de Berkeley trabaja con los *valores de tiempo reportados* por cada cliente, no con el momento exacto en que esos valores llegan al servidor.

   Sin embargo, la latencia sí afecta la **precisión** del resultado. Si un cliente tiene alta latencia, el tiempo que reporta ya estará "desactualizado" cuando llegue al servidor, introduciendo un error sistemático. En nuestra implementación, el cliente captura su tiempo local *antes* de la latencia, por lo que el ajuste calculado es correcto para ese instante, pero cuando el cliente finalmente aplica el ajuste, su reloj real ya avanzó.

2. **¿Por qué el algoritmo de Berkeley es sensible o no a este factor?**

   El algoritmo de Berkeley es **parcialmente sensible** a la latencia por las siguientes razones:

   Insensibilidad: El algoritmo no depende de una fuente de tiempo externa (como NTP), sino que calcula un promedio democrático de todos los participantes. La latencia no impide que el algoritmo calcule y distribuya los ajustes correctamente.

   Sensibilidad: En redes con alta latencia o latencia asimétrica, el algoritmo puede introducir errores porque:

   - El tiempo que tarda un mensaje en llegar no se compensa (a diferencia del algoritmo de Cristian que usa RTT/2).

- Si la latencia varía significativamente entre clientes, algunos recibirán sus ajustes más tarde, acumulando desfase adicional.

Para mitigar esto, implementaciones robustas de Berkeley incorporan estimación de RTT (Round-Trip Time) y compensan la latencia al calcular los ajustes.

1.2.4. Actividad 5

Modificar la implementación anterior para ejecutar, servidor y clientes en máquinas diferentes.

1.3. Algoritmo de Lamport:

Para la implementación del algoritmo de relojes lógicos de Lamport se proporciona un programa en Python que utiliza el módulo multiprocessing para la generación de procesos locales.

1.3.1. Actividad 6

- El programa es de acuerdo a un ambiente de eventos establecido de cada proceso.
- Se pide probarlo y entenderlo para poder plantear en equipo un ambiente sugerido con 7 procesos.
- Entregar diagrama de ambiente propuesto

Para la implementación del algoritmo de relojes lógicos de Lamport, se analizó inicialmente un programa base en Python que utiliza el módulo `multiprocessing` para la generación de procesos locales de forma concurrente, usando tuberías (`Pipe`) para la comunicación entre ellos.

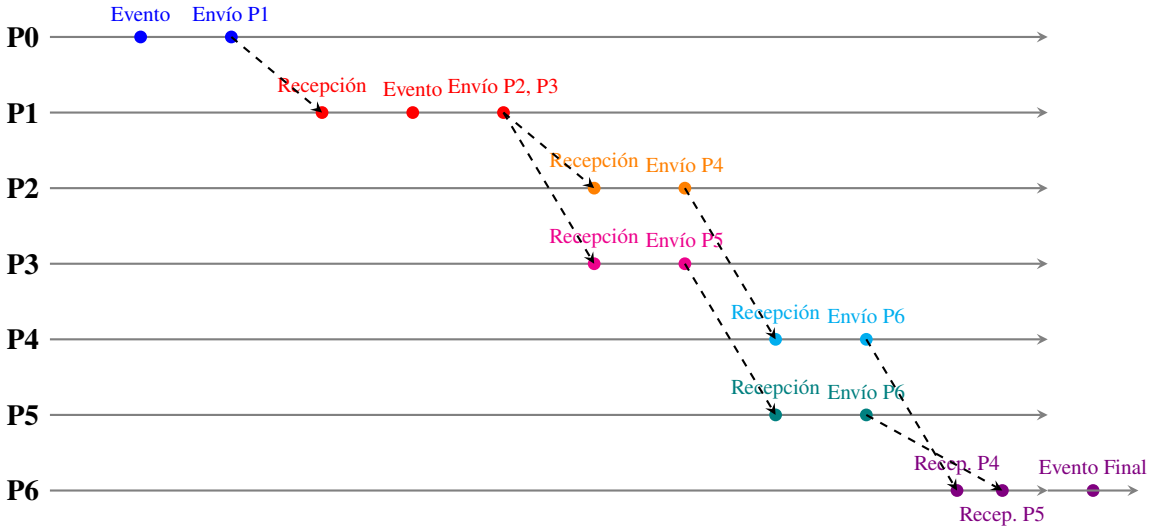
El objetivo principal de esta actividad fue probar y comprender el funcionamiento de este mecanismo de sincronización basado en el paso de mensajes y eventos locales, para posteriormente plantear, diseñar e implementar un **ambiente sugerido por 7 procesos**. Dado que se requiere que cada ambiente sea diferente y original, se diseñó una topología en cascada con ramificaciones y convergencia.

El ambiente diseñado para esta simulación tiene siete procesos (P0 a P6) que interactúan de la siguiente manera, asegurando que el reloj de Lamport siga funcionando bien cuando el sistema crece, incluso si las conexiones entre los nodos no son simples o en línea recta.

- **P0:** Inicia la cascada ejecutando un evento local y enviando un mensaje a P1.
- **P1:** Recibe el mensaje de P0, procesa un evento local, y envía mensajes hacia dos procesos distintos: P2 y P3.
- **P2 y P3:** Reciben los mensajes de P1 de manera paralela. P2 envía su mensaje a P4, mientras que P3 envía su mensaje a P5.
- **P4 y P5:** Reciben la información de sus respectivos emisores y la reenvían hacia el nodo final, P6.
- **P6:** Actúa como nodo colector. Recibe los mensajes tanto de P4 como de P5, actualizando su reloj de Lamport según las estampas de tiempo recibidas, y finaliza con un evento local.

Diagrama del Ambiente Propuesto

A continuación, se presenta el diagrama propuesto que modela el intercambio de mensajes y la secuencia de eventos del ambiente propuesto.



Implementación del Código

El siguiente código implementa el ambiente propuesto de 7 procesos utilizando la regla de actualización de relojes de Lamport: $C_j = \text{máx}(C_j, C_m) + 1$. Para mejorar la depuración, se incluyó código de color ANSI en la salida estándar, lo que permite diferenciar visualmente la naturaleza de cada acción.

Listing 1: Implementación de Lamport con 7 Procesos

```python
from multiprocessing import Process, Pipe
from os import getpid
from datetime import datetime

# ==========================================
# CÓDIGOS DE COLOR ANSI PARA LA TERMINAL
# ==========================================
RESET = "\033[0m"
C_EVENT = "\033[93m"
C_SEND  = "\033[92m"
C_RECV  = "\033[96m"
C_TIME  = "\033[90m"

def local_time(counter):
    time_str = datetime.now().strftime('%H:%M:%S.%f')[:-3]
    return f'{C_TIME}[ Reloj Lamport: {counter:02d} | Hora: {time_str} ]{RESET}'

def calc_recv_timestamp(recv_time_stamp, counter):
    return max(recv_time_stamp, counter) + 1
```

```python

def event(pid, counter, p_num):
    counter += 1
    accion = f'{C_EVENT}[P{p_num}] EVENTO LOCAL   (PID:{pid}){RESET}'.ljust(52)
    print(f'{accion} {local_time(counter)}')
    return counter

def send_message(pipe, pid, counter, p_num, dest_num):
    counter += 1
    pipe.send(('Mensaje', counter))
    accion = f'{C_SEND}[P{p_num}] ENVÍA a P{dest_num}    (PID:{pid}){RESET}'.ljust(52)
    print(f'{accion} {local_time(counter)}')
    return counter

def recv_message(pipe, pid, counter, p_num, src_num):
    message, timestamp = pipe.recv()
    counter = calc_recv_timestamp(timestamp, counter)
    accion = f'{C_RECV}[P{p_num}] RECIBE de P{src_num}  (PID:{pid}){RESET}'.ljust(52)
    print(f'{accion} {local_time(counter)}')
    return counter

# ==========================================
# DEFINICIÓN DE LOS 7 PROCESOS
# ==========================================
def p0(pipe01):
    pid = getpid()
    counter = 0
    counter = event(pid, counter, 0)
    counter = send_message(pipe01, pid, counter, 0, 1)

def p1(pipe10, pipe12, pipe13):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe10, pid, counter, 1, 0)
    counter = event(pid, counter, 1)
    counter = send_message(pipe12, pid, counter, 1, 2)
    counter = send_message(pipe13, pid, counter, 1, 3)

def p2(pipe21, pipe24):
```

```
    pid = getpid()
    counter = 0
    counter = recv_message(pipe21, pid, counter, 2, 1)
    counter = send_message(pipe24, pid, counter, 2, 4)

def p3(pipe31, pipe35):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe31, pid, counter, 3, 1)
    counter = send_message(pipe35, pid, counter, 3, 5)

def p4(pipe42, pipe46):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe42, pid, counter, 4, 2)
    counter = send_message(pipe46, pid, counter, 4, 6)

def p5(pipe53, pipe56):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe53, pid, counter, 5, 3)
    counter = send_message(pipe56, pid, counter, 5, 6)

def p6(pipe64, pipe65):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe64, pid, counter, 6, 4)
    counter = recv_message(pipe65, pid, counter, 6, 5)
    counter = event(pid, counter, 6)

if __name__ == '__main__':
    print("\n" + "="*70)
    print(" INICIANDO SIMULACIÓN DE RELOJES DE LAMPORT (7 PROCESOS) ".center
        (70, "="))
    print("="*70 + "\n")

    # Creación de tuberías para comunicación
    pipe01, pipe10 = Pipe()
    pipe12, pipe21 = Pipe()
    pipe13, pipe31 = Pipe()
    pipe24, pipe42 = Pipe()
    pipe35, pipe53 = Pipe()
```

```
    pipe46, pipe64 = Pipe()
    pipe56, pipe65 = Pipe()

    # Inicialización
    procesos = [
        Process(target=p0, args=(pipe01,)),
        Process(target=p1, args=(pipe10, pipe12, pipe13)),
        Process(target=p2, args=(pipe21, pipe24)),
        Process(target=p3, args=(pipe31, pipe35)),
        Process(target=p4, args=(pipe42, pipe46)),
        Process(target=p5, args=(pipe53, pipe56)),
        Process(target=p6, args=(pipe64, pipe65))
    ]

    for p in procesos:
        p.start()
    for p in procesos:
        p.join()

    print("\n" + "="*70)
    print(" SIMULACIÓN FINALIZADA ".center(70, "="))
    print("="*70 + "\n")
```

El código implementa el algoritmo de sincronización de Lamport usando la librería `multiprocessing` de Python, la cual permite ejecutar los 7 procesos de manera verdaderamente concurrente, cada uno con su propio espacio de memoria y su propio Identificador de Proceso (PID).

1. Reglas Lógicas del Reloj de Lamport

El núcleo del algoritmo se divide en tres funciones fundamentales que alteran la variable `counter` (el reloj lógico local de cada proceso):

- **Eventos Locales (`event`):** Simplemente incrementan el reloj local en 1 unidad ($C_i = C_i + 1$).
- **Envío de Mensajes (`send_message`):** Antes de enviar, el proceso incrementa su reloj local en 1 ($C_i = C_i + 1$) y luego adjunta ese valor actualizado (estampa de tiempo) dentro de la tubería junto con el mensaje mediante `pipe.send(('Mensaje', counter))`.
- **Recepción de Mensajes (`recv_message`):** Cuando un proceso extrae un mensaje de la tubería mediante `pipe.recv()`, lee la estampa de tiempo adjunta. Es aquí donde se aplica la regla de convergencia principal de Lamport implementada en la función `calc_recv_timestamp`: el nuevo tiempo del proceso receptor será el valor máximo entre su propio reloj local y la estampa de tiempo recibida del

mensaje, más uno ($C_j = \text{máx}(C_j, C_m) + 1$).

2. Cumplimeiento del ambiente propuesto

La comunicación entre procesos se logra mediante objetos `Pipe()`, que actúan como canales bidireccionales. Para garantizar que el programa se comporte exactamente como el diagrama porpuesto, las conexiones y llamadas a las funciones se estructuraron de la siguiente forma:

- **El Inicio (P0):** La función `p0` arranca el sistema ejecutando un `event()` seguido de un `send_message()` usando `pipe01`. Esto coincide con el origen de la cascada.
- **La Ramificación (P1):** La función `p1` está diseñada para bloquearse hasta recibir el mensaje de P0 mediante `recv_message(pipe10)`. Una vez que el reloj se actualiza, P1 ejecuta un evento local y luego se divide, llamando a `send_message` dos veces: una hacia P2 (`pipe12`) y otra hacia P3 (`pipe13`).
- **El Paralelismo (P2, P3, P4, P5):** Las funciones `p2` y `p3` funcionan como intermediarios en paralelo. Al recibir el mensaje de P1, envían su propio mensaje hacia P4 y P5 respectivamente. A su vez, `p4` y `p5` reciben esos mensajes y los retransmiten hacia el colector final. Esto asegura que la estampa de tiempo siga arrastrando la historia causal de los eventos previos de manera paralela.
- **La Convergencia (P6):** La función `p6` consolida la información de las dos ramas. Realiza dos llamadas a `recv_message`, extrayendo los mensajes de `pipe64` y `pipe65`. La función `calc_recv_timestamp` internamente se asegurará de que el reloj de P6 absorba el tiempo más grande que le llegue de ambas ramas. Finalmente, realiza un último `event()` para marcar la conclusión del flujo.

Este diseño refleja fielmente las flechas trazadas en el diagrama de ambiente propuesto, forzando a los procesos a respetar la relación "sucedió antes que" ($\rightarrow$) diseñada para esta topología de 7 nodos y validando el correcto funcionamiento de los relojes lógicos.

Funcionamiento del código

Al ejecutar el programa, pudimos comprobar que los relojes de Lamport funcionan correctamente. Notamos que, aunque los mensajes se impriman en distinto orden en la consola (ya que la computadora ejecuta los procesos al mismo tiempo), los tiempos lógicos siempre se mantienen organizados.

Lo más importante que observamos es que el reloj lógico del proceso final (`P6`) siempre termina con un valor mayor al de todos los procesos que le enviaron información. Esto demuestra que el sistema cumple a la perfección con la regla de "sucedió antes que" (*happened-before*) de Leslie Lamport: si un evento ocurre antes que otro, su marca de tiempo siempre será menor.

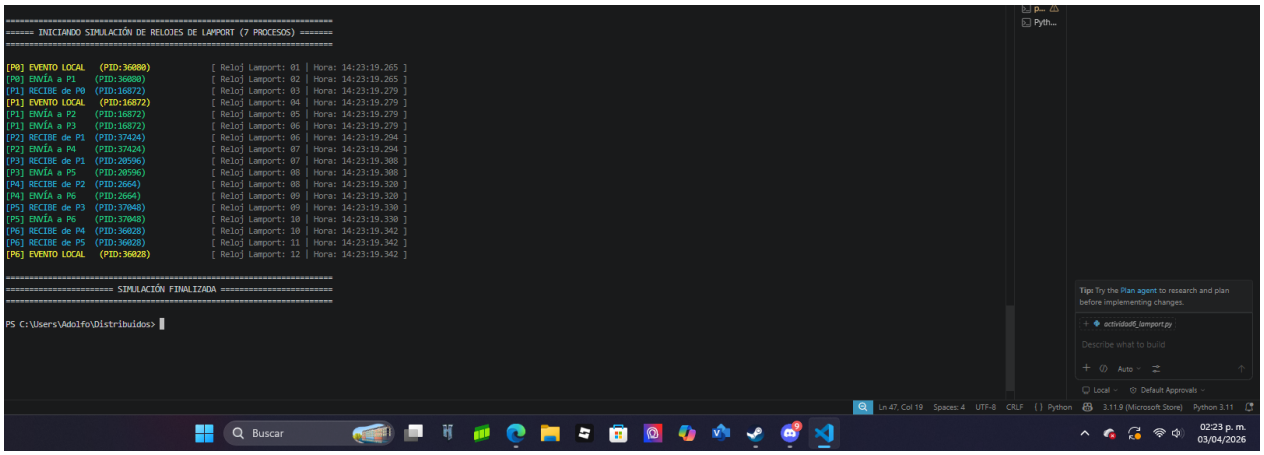


Figura 1: Simulación de relojes de Lamport con 7 procesos

1.3.2. Actividad 7

Con la idea anterior implementar una solución al algoritmo de relojes vectoriales, plantear un ambiente con 5 procesos. Entregar la figura con el ambiente sugerido que se programará.

Para esta actividad, se adaptó la arquitectura concurrente basada en `multiprocessing` y `Pipe` de la Actividad 6. La diferencia principal es que, en lugar de usar un único contador entero (Lamport), cada proceso mantiene un arreglo (vector) con el registro del tiempo lógico de todos los participantes del sistema. Al ser 5 procesos, el reloj de cada uno es una lista de 5 posiciones: `[P0, P1, P2, P3, P4]`.

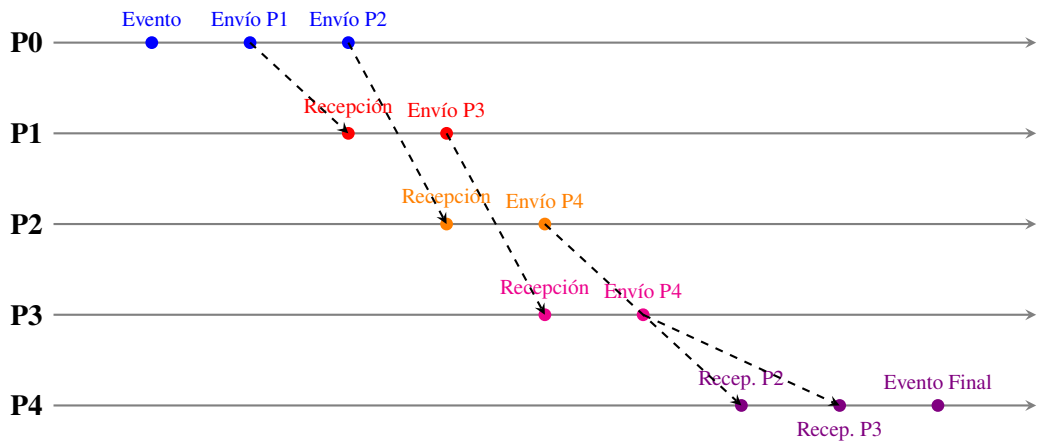
El objetivo fue crear un sistema con 5 procesos donde la información se divide en varios caminos y luego se vuelve a juntar, para ver cómo cada proceso va guardando y combinando la información de lo que pasó en los distintos caminos.

El ambiente diseñado interactúa de la siguiente manera:

- **P0:** Inicia la ejecución con un evento local. Posteriormente, envía un mensaje a P1 y luego otro mensaje a P2.
- **P1:** Recibe el mensaje de P0, actualiza su vector y envía un mensaje a P3.
- **P2:** Recibe el mensaje de P0 de forma paralela a P1, actualiza su vector y envía un mensaje a P4.
- **P3:** Recibe el mensaje de P1, procesa la información y reenvía su mensaje a P4.
- **P4:** Actúa como el nodo de convergencia. Primero recibe el mensaje de P2, luego recibe el mensaje de P3 (uniendo ambas ramas), y finaliza su ejecución con un evento local.

Diagrama del Ambiente Propuesto

A continuación, se presenta el diagrama propuesto que modela el intercambio de mensajes y eventos para el ambiente de 5 procesos.



Implementación del Código

El siguiente código implementa el algoritmo de relojes vectoriales para los 5 procesos. La regla de actualización principal al recibir un mensaje toma el máximo valor posición por posición entre el reloj local y el recibido.

Listing 2: Implementación de Relojes Vectoriales con 5 Procesos

```
from multiprocessing import Process, Pipe
from os import getpid
from datetime import datetime

# ========================================
# CÓDIGOS DE COLOR ANSI PARA LA TERMINAL
# ========================================
RESET = "\033[0m"
C_EVENT = "\033[93m"
C_SEND  = "\033[92m"
C_RECV  = "\033[96m"
C_TIME  = "\033[90m"

def local_time(counter):
    time_str = datetime.now().strftime('%H:%M:%S.%f')[:-3]
    vector_str = f"[{', '.join(f'{x:02d}' for x in counter)}]"
    return f'{C_TIME}[ Reloj Vectorial: {vector_str} | Hora: {time_str} ]{
        RESET}'
```

```python

def calc_recv_timestamp(recv_time_stamp, counter):
    return [max(r, c) for r, c in zip(recv_time_stamp, counter)]

def event(pid, p_index, counter):
    counter[p_index] += 1
    accion = f'{C_EVENT}[P{p_index}] EVENTO LOCAL   (PID:{pid}){RESET}'.ljust(52)
    print(f'{accion} {local_time(counter)}')
    return counter

def send_message(pipe, pid, p_index, counter, dest_num):
    counter[p_index] += 1
    pipe.send(('Mensaje', counter))
    accion = f'{C_SEND}[P{p_index}] ENVÍA a P{dest_num}    (PID:{pid}){RESET}'.ljust(52)
    print(f'{accion} {local_time(counter)}')
    return counter

def recv_message(pipe, pid, p_index, counter, src_num):
    message, timestamp = pipe.recv()
    counter = calc_recv_timestamp(timestamp, counter)
    counter[p_index] += 1
    accion = f'{C_RECV}[P{p_index}] RECIBE de P{src_num}  (PID:{pid}){RESET}'.ljust(52)
    print(f'{accion} {local_time(counter)}')
    return counter

# ==========================================
# DEFINICIÓN DE LOS 5 PROCESOS
# ==========================================
def p0(pipe01, pipe02):
    pid = getpid()
    counter = [0, 0, 0, 0, 0]
    counter = event(pid, 0, counter)
    counter = send_message(pipe01, pid, 0, counter, 1)
    counter = send_message(pipe02, pid, 0, counter, 2)

def p1(pipe10, pipe13):
    pid = getpid()
    counter = [0, 0, 0, 0, 0]
    counter = recv_message(pipe10, pid, 1, counter, 0)
```

```
    counter = send_message(pipe13, pid, 1, counter, 3)

def p2(pipe20, pipe24):
    pid = getpid()
    counter = [0, 0, 0, 0, 0]
    counter = recv_message(pipe20, pid, 2, counter, 0)
    counter = send_message(pipe24, pid, 2, counter, 4)

def p3(pipe31, pipe34):
    pid = getpid()
    counter = [0, 0, 0, 0, 0]
    counter = recv_message(pipe31, pid, 3, counter, 1)
    counter = send_message(pipe34, pid, 3, counter, 4)

def p4(pipe42, pipe43):
    pid = getpid()
    counter = [0, 0, 0, 0, 0]
    counter = recv_message(pipe42, pid, 4, counter, 2)
    counter = recv_message(pipe43, pid, 4, counter, 3)
    counter = event(pid, 4, counter)

if __name__ == '__main__':
    print("\n" + "="*80)
    print(" INICIANDO SIMULACIÓN DE RELOJES VECTORIALES (5 PROCESOS) ".center
        (80, "="))
    print("="*80 + "\n")

    pipe01, pipe10 = Pipe()
    pipe02, pipe20 = Pipe()
    pipe13, pipe31 = Pipe()
    pipe24, pipe42 = Pipe()
    pipe34, pipe43 = Pipe()

    procesos = [
        Process(target=p0, args=(pipe01, pipe02)),
        Process(target=p1, args=(pipe10, pipe13)),
        Process(target=p2, args=(pipe20, pipe24)),
        Process(target=p3, args=(pipe31, pipe34)),
        Process(target=p4, args=(pipe42, pipe43))
    ]

    for p in procesos:
```

```
        p.start()
    for p in procesos:
        p.join()

    print("\n" + "="*80)
    print(" SIMULACIÓN FINALIZADA ".center(80, "="))
    print("="*80 + "\n")
```

Al igual que en el algoritmo anterior, usamos `multiprocessing` para garantizar la ejecución concurrente, pero el manejo del tiempo cambia significativamente.

1. Reglas Lógicas del Reloj Vectorial

Cada proceso inicia con un arreglo de ceros: `[0, 0, 0, 0, 0]`. La posición `i` del arreglo corresponde al conocimiento que tiene el proceso sobre el tiempo del proceso `i`.

- **Eventos Locales y Envíos:** Cada vez que un proceso `Pi` hace un evento o envía un mensaje, incrementa únicamente su propia posición en el vector (`counter[i] += 1`). En el caso de envío, manda el arreglo completo por la tubería.
- **Recepción de Mensajes:** Cuando `Pi` recibe un mensaje, compara su vector actual con el vector recibido. Usa la función `max()` **posición por posición** para quedarse con el valor más grande de cada índice. Finalmente, incrementa su propio índice para registrar el evento de recepción.

2. Cumplimiento del ambiente propuesto

La estructura de las tuberías (`Pipe`) sigue fielmente el diagrama diseñado:

- **P0** tiene su vector inicial y tras sus tres acciones (un evento local y dos envíos) su índice 0 avanza, propagando esos valores a P1 y P2.
- **P1 y P2** reciben la historia de P0. Al actualizar sus vectores, adoptan el valor del índice 0 que traía P0, y luego incrementan sus respectivos índices (1 y 2) para retransmitirlos a los siguientes procesos.
- **P3** actúa como puente, absorbiendo la historia de P1 (que ya incluía a P0) y enviándola a P4.
- **P4 (Convergencia):** Es el punto clave de la simulación. Al recibir información de P2 y de P3, su función `calc_recv_timestamp` combina ambas historias usando los máximos. Esto le permite a P4 tener una "fotografía" completa de cuántos eventos ocurrieron en cada uno de los procesos anteriores.

Funcionamiento del código

Al analizar los resultados de la consola , podemos comprobar paso a paso cómo los relojes vectoriales preservan toda la historia causal. A diferencia de Lamport, que solo daba un número general, aquí podemos ver exactamente qué sabe cada proceso sobre el resto del sistema.

Observando la ejecución, el proceso **P0** inicia la cascada: tras su primer evento local y sus envíos a P1 y P2, vemos cómo únicamente incrementa su propia posición (la posición 0 del arreglo), llegando hasta el vector `[03, 00, 00, 00, 00]`.

Cuando los demás procesos reciben mensajes, podemos notar cómo absorben esta historia. Por ejemplo, cuando **P1** recibe el mensaje de P0, compara el vector recibido con el suyo y actualiza su reloj a `[02, 01, 00, 00, 00]`, adoptando la información de los 2 eventos que llevaba P0 en ese instante y sumando el suyo propio. De igual forma, cuando **P2** recibe su mensaje, su vector se actualiza a `[03, 00, 01, 00, 00]`.

La ventaja principal de este algoritmo se demuestra en el nodo de convergencia, **P4**. En la imagen, vemos que P4 recibe primero el mensaje de P2, actualizando su vector a `[03, 00, 02, 00, 01]`. Inmediatamente después, recibe el mensaje de P3 (el cual trae un vector de `[02, 02, 00, 02, 00]`). Al aplicar la regla de tomar el valor máximo posición por posición entre ambos vectores, P4 fusiona exitosamente ambas ramas.

Tras su evento local final, P4 concluye con el vector `[03, 02, 02, 02, 03]`. Este arreglo final nos dice con exactitud que, antes de que P4 terminara, ocurrieron 3 eventos en P0, 2 en P1, 2 en P2 y 2 en P3. Esto demuestra que la topología funcionó perfectamente y que no hubo ninguna pérdida de información en todo el sistema.

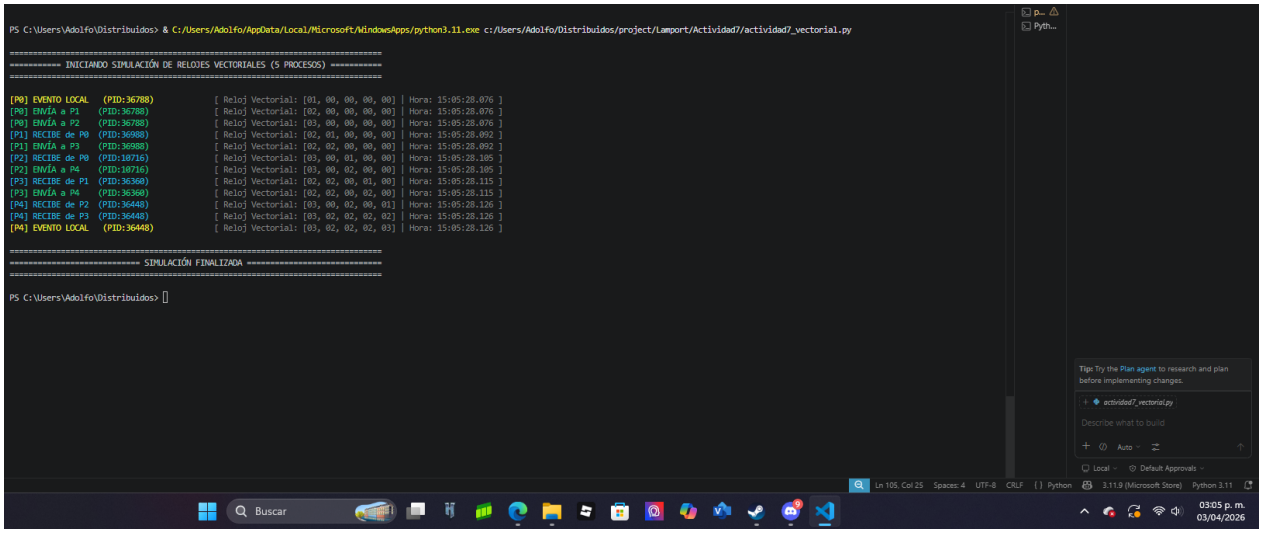


Figura 2: Simulación de relojes Vectoriales con 5 procesos

Referencias

Oracle. (s.f.). *Oracle VM VirtualBox - Downloads | Oracle Technology Network | Oracle*. https://www.oracle.com/virtualization/technologies/vm/downloads/virtualbox-downloads.html

Petrosez. (2025, 12 de septiembre). *PelicanHPC*. https://sourceforge.net/projects/pelicanhpc/

Saenz García, E. K. (2026, 3 de marzo). *ClusterHPC*. YouTube. https://www.youtube.com/watch?v=F_LFf0lhbRA